

CS482/495/496 Software Project: add your tentative project title here

your name(s) here

2026-10-08

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Dr. Rzeszotarski
- Client title: Assistant Professor
- Client email address: jrzeszotarski@loyola.edu
- Client employer: Loyola University Maryland

2 Project Description

2.1 Overview

The project is a web application for playing Caribbean Stud poker with virtual money, it will provide a way for users to learn the game without putting down real money. Players can create or join public and private lobbies, while guests can play without creating an account. An automated dealer will manage each game.

The Client wants a Caribbean-themed poker website that will attract players and provide opportunities for advertising revenue. The app will also have admin tools for managing users, distributing money bonuses, and reviewing reports.

2.2 Key Features

1. Caribbean Stud Gameplay

- Implement Caribbean Stud according to the rules and edge cases.
- Include a dealer bot.
- Support games with 2–6 players.
- Allow lobby owners to enable or disable bot players.
- Provide an optional heads-up display showing the current hand's chance of winning.

2. Player Accounts and Guest Access

- Allow users to create accounts with usernames, avatars, email login, and passwords.
- Support password resets.
- Track player wins, losses, and fake-money.
- Allow guests to play without creating an account.

- Allow guests to register after playing and retain their current fake-money balance.

3. Lobby System

- Allow players to create and join public and private lobbies (password protected).
- Allow lobby owners to configure the number of players, bots, minimum and maximum bets, minimum money requirements, and privacy settings.
- Store lobby information including the owner and creation timestamp.
- Provide an optional automatic minimum-bet increase as players gain money.
- Mark lobbies inactive after a period with zero players.

4. Spectating and Reconnection

- Allow users to spectate public and private games according to the lobby's access settings.
- Support an unlimited number of spectators.
- Track each player's active games.
- Automatically reconnect returning players to their previous lobby or game.
- Give disconnected players a 15–30 second period to return before automatically losing the hand.

5. Fake-Money System

- Give registered users a starting amount of fake money.
- Provide a top-up feature for obtaining additional fake money.
- Support administrator-triggered bonuses, such as a 100-point holiday bonus for all players.

6. Social Features

- Provide text chat within game lobbies.
- Allow players to add friends.
- Allow players to invite friends directly to a lobby.
- Display player usernames and avatars within lobbies.

7. Tutorial and User Experience

- Provide an interactive tutorial explaining how to play Caribbean Stud.
- Use a light-blue/cyan color palette with a warm Caribbean-inspired theme.
- Include an 18+ and responsible-play disclaimer.

8. Administration

- Provide a contact-admin feature for users to report issues.
- Allow administrators to view reported issues.
- Allow administrators to ban users.
- Provide an analytics dashboard showing rooms created, active players, and ad impressions.

9. Security and Game Integrity

- Protect player accounts from unauthorized access.
- Protect the game from cheating and client-side manipulation.
- Address fake-account exploits.
- Securely hash user passwords.

10. Stretch Goals

- Restrict guests from using fake-money top-ups.
- Limit registered players to 2–3 free top-ups before requiring an advertisement for another top-up.
- Allow administrators to monitor previously played hands.

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?] hk

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
----	-------------	--------	-------------

S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.
Total:		80	

Table 2: Functional requirements as User Stories.

3.3 AI Analysis on Functional Requirements

[After you receive your revised User Stories in CS482, or after your first delivery of Stories on CS496, you should use AI as a senior software engineer to help you find any missing or hidden requirements.] [You must provide which LLM you use, which prompt you used, and most importantly **reflect** on the results the LLM gave back. Just because the LLM suggested something, it does not mean it was a valid suggestion and that you should follow blindly. Use your knowledge and reason on which ones are reasonable to add to your requirements and which ones are not.]

4 System Design

4.1 Architecture

Our team is following a Layered Client Server architecture based on MVC principles. The application will be separated into a presentation layer for the user interface, a business Logic layer to handle the core game loops, and a data access layer to manage real time database synchronization.

Main Modules: Grouped by semantic responsibility, our main modules are:

User & Account Module: Manages player registration, guest sessions, authentication, and user profiles.

Lobby & Matchmaking Module: Handles public and private room creation, spectator access, and session tracking to allow for reconnections.

Game Engine Module: Enforces the exact Caribbean Stud rules, operates the dealer bot, and manages betting mechanics like the auto incrementing minimum bet.

Economy Module: Controls fake money balances, handles the top-up system, and manages ad watch restrictions.

Admin & Security Module: Provides tools for user moderation (bans, password resets), triggering global bonuses, handling issue reports, and viewing the analytics dashboard.

4.2 Diagrams

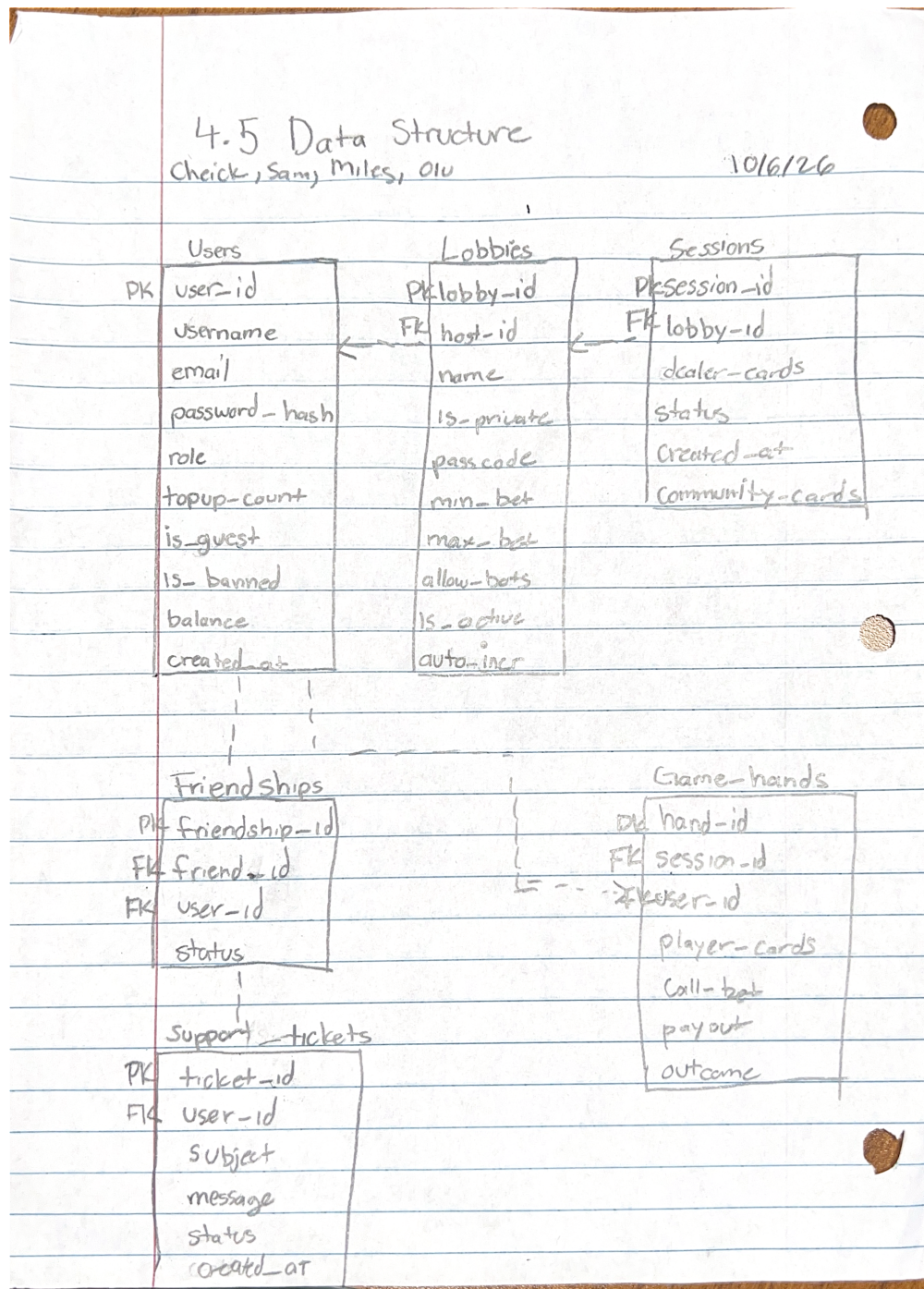


Figure 1: Data Structure

4.3 Technology

- Real-time updates: Socket.IO - handles lobby chat, gameplay, reconnections. Essentially, creates a server
- Frontend: React + TypeScript to build the game interface, forms, and lobbies

- Backend: Node.js + Express: Account management, lobby settings, game rules, admin functions. The sever controls card dealing, bet validation, and payouts.
- Database: PostgreSQL: stores user accounts, lobby settings, game history, chip balances.
- Testing: Vitest for game rules, payouts, and any backend logic. Playwright to test the overall usability in the web browser(logging in, joining a lobby, playing a hand)
- GitHub Actions: Build will run and be tested on each pull request. All required checks must pass before changes can be merged into the main branch.

4.4 Coding Standards

Yes, our team will follow coding standards, specifically with variable naming. All variables and functions will use camelCase. Database tables will be plural. We will also only allow code with unit tests and above 50% coverage to be committed.

4.5 Data

For this project, we are using a relational SQL database. To ensure clean data architecture, all user roles are consolidated into a single table, as they share the same core authentication and profile fields.

Main Tables and Attributes

- **Users:** `user_id` (PK), `role`, `username`, `email`, `password`, `avatar`, `balance`, `wins`, `losses`, `top-ups`, `is_banned`.
- **Lobbies:** `lobby_id` (PK), `owner_id` (FK), `is_private`, `password`, `max_players`, `bots_on`, `min_bet`, `max_bet`, `min_money`, `auto_bet`, `is_active`.
- **Lobby_Participants:** `lobby_id` (FK), `user_id` (FK), `is_spectator`, `last_active`.
- **Chat_Messages:** `message_id` (PK), `lobby_id` (FK), `sender_id` (FK), `text`, `timestamp`.
- **Friendships:** `user_id_1` (FK), `user_id_2` (FK), `status`.
- **Support_Reports:** `report_id` (PK), `reporter_id` (FK), `issue`, `status`.
- **Hand_Logs :** `hand_id` (PK), `lobby_id` (FK), `cards`, `winner_id` (FK), `timestamp`.

ERD Interactions

- **Users ↔ Lobbies (1:N):** A User can own multiple Lobbies, but a Lobby has only one owner.
- **Users ↔ Lobbies via Participants (M:N):** Users can join multiple lobbies, and lobbies hold multiple users. The `Lobby_Participants` table bridges this connection and tracks active timeouts.
- **Users ↔ Users (M:N):** The `Friendships` table links a User to multiple other User friends.
- **Lobbies ↔ Chat (1:N):** A single Lobby contains multiple Chat Messages.
- **Users ↔ Support (1:N):** A User can submit multiple issue reports to the admins.

4.6 UI Mocks

The UI mockups show the main interfaces of ReefRoyale. These screenshots show how users will navigate the site and create/join lobbies.

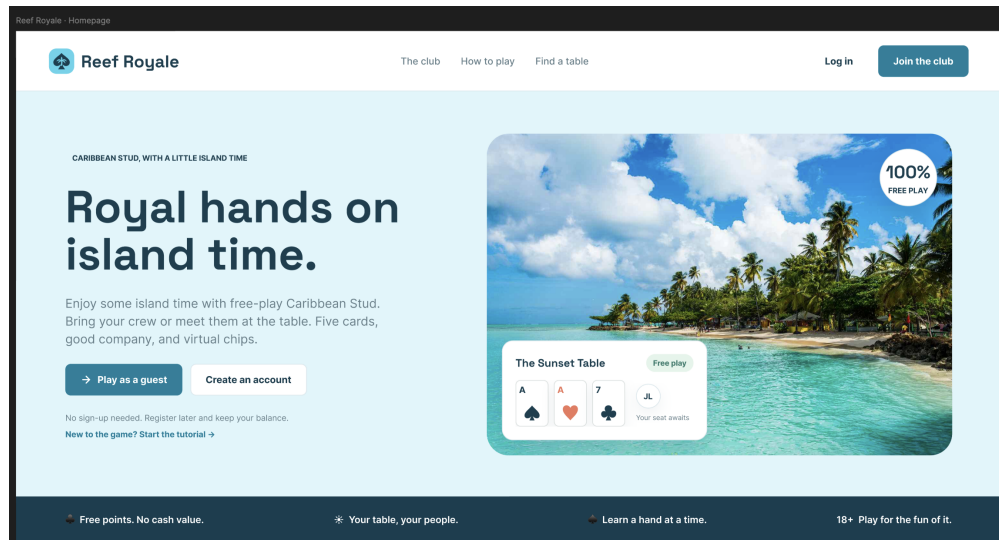


Figure 2: ReefRoyale Home Page: The main landing page where users can navigate to different features of the application.

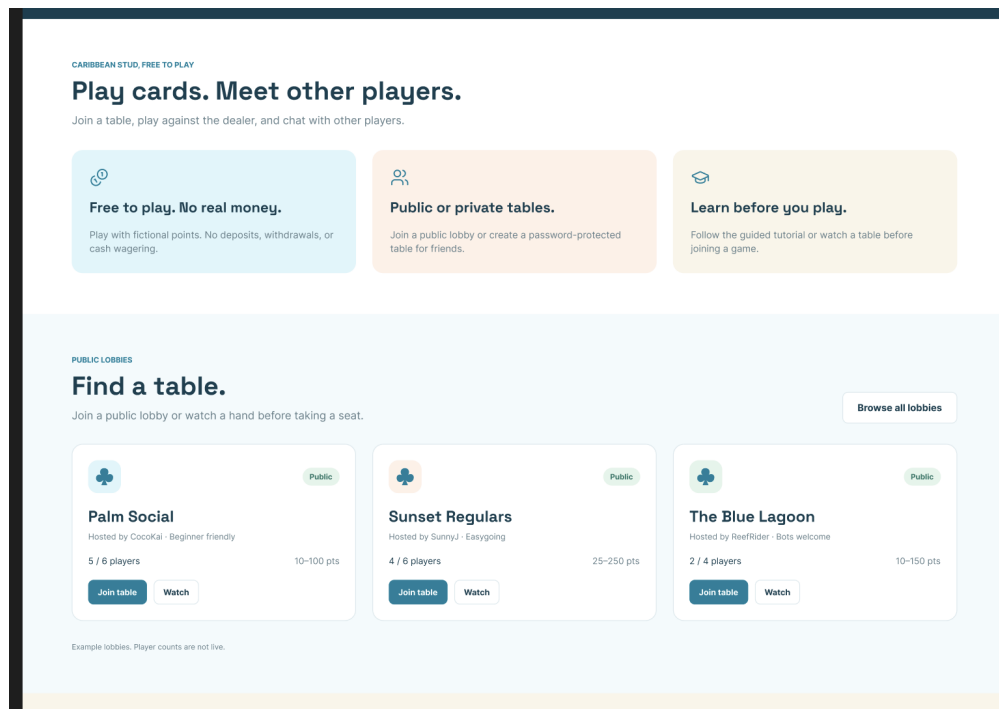


Figure 3: ReefRoyale UI Mockup: Additional application interface displaying game features and navigation options.

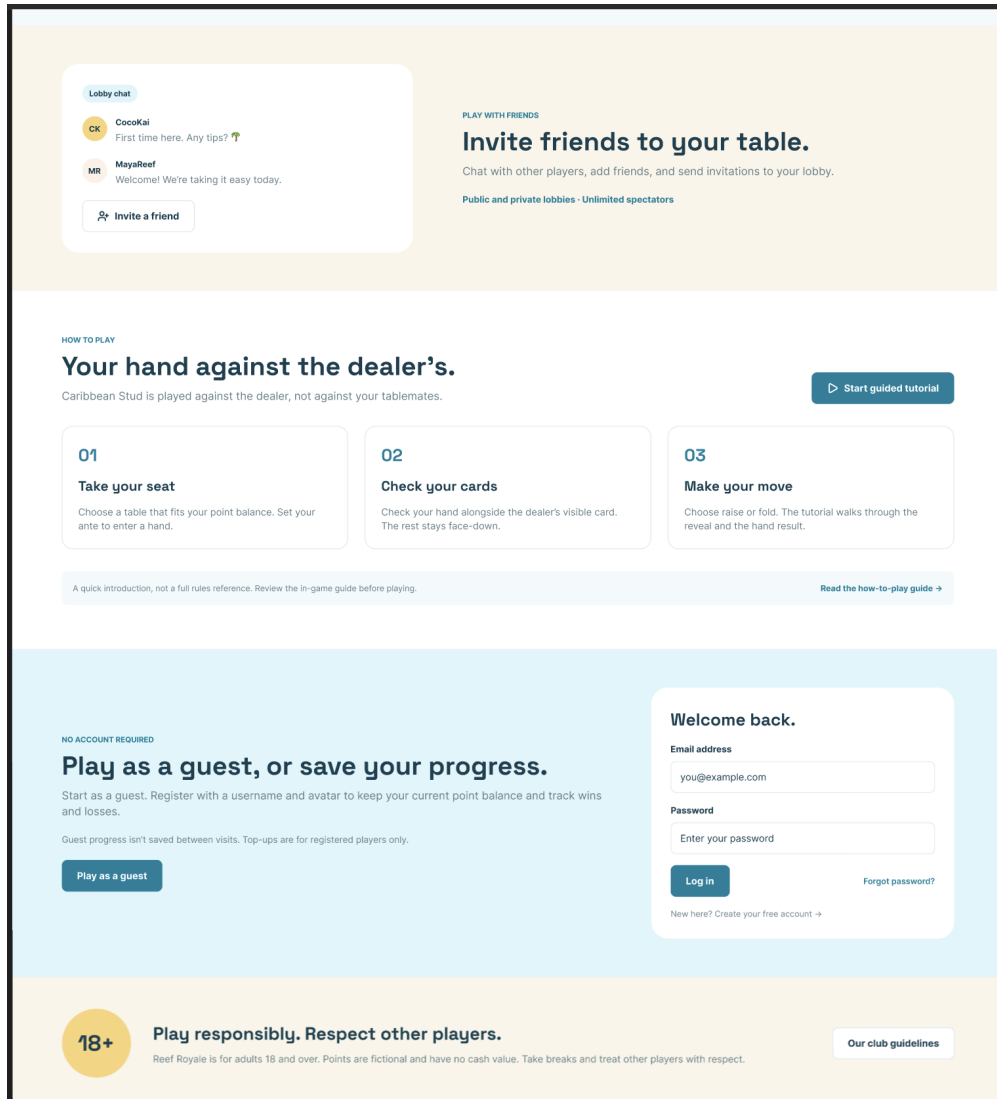


Figure 4: ReefRoyale UI Mockup: Instructions on how to play the game, and basic info.

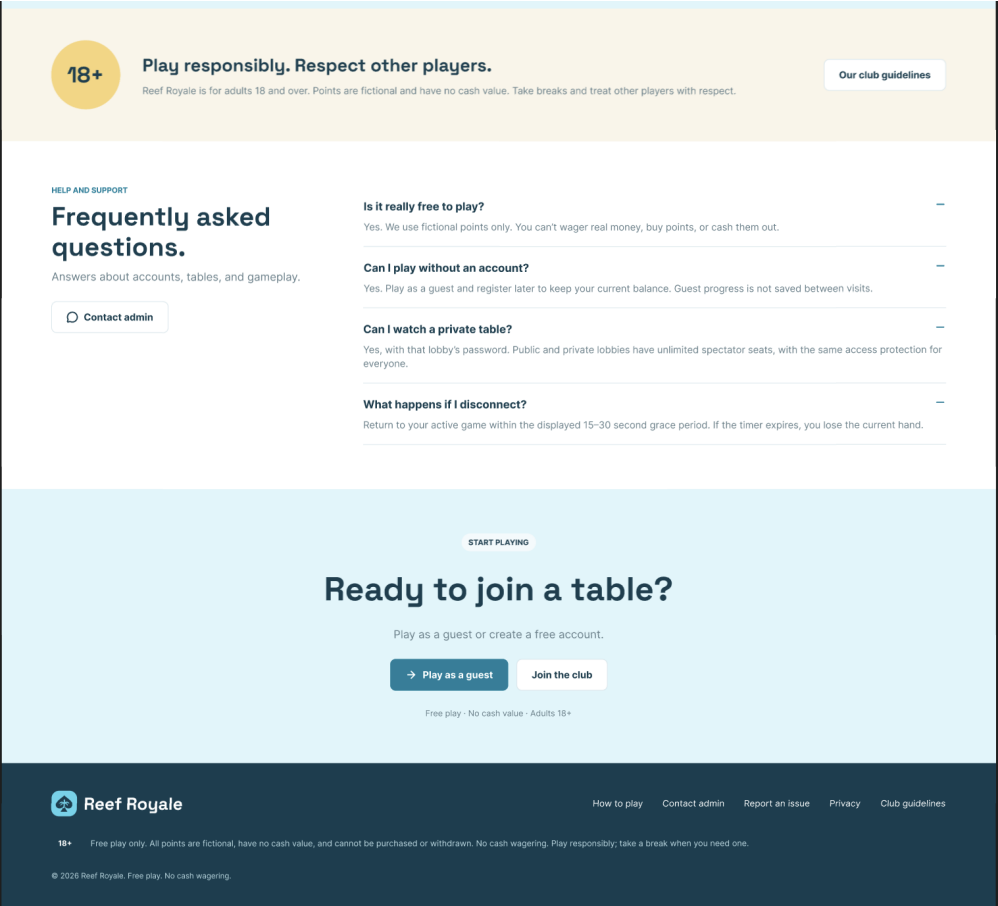


Figure 5: ReefRoyale UI Mockup: Frequently Asked Questions

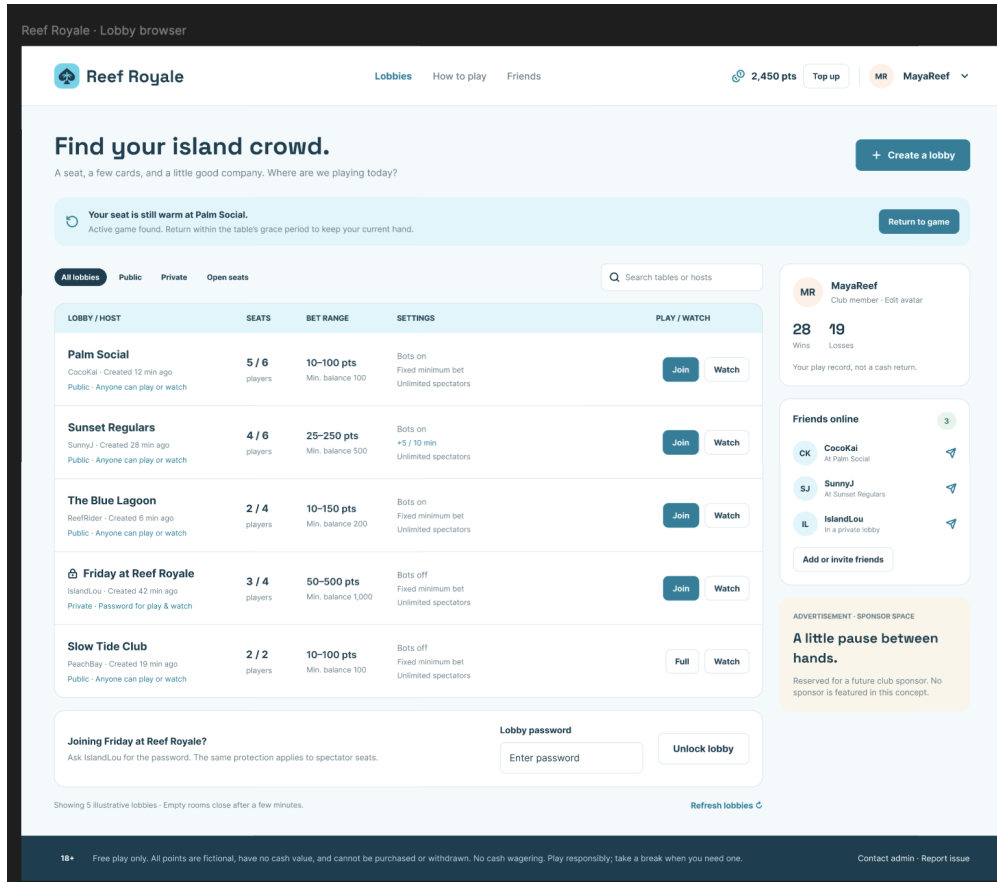


Figure 6: Lobby Browser: Allows players to browse available game lobbies and select one to join.

Reef Royale · Create a lobby

Reef Royale

Lobbies
How to play
Friends

2,450 pts
Top up
MR
MayaReef

Back to lobbies

Your table. Your kind of company.

Set the pace, choose your people, and let the dealer take it from there.

FREE-PLAY LOBBY

01
Give your table a name

Something your friends can spot in the lobby browser.

Lobby name

Maya's Island Hangout

3-40 characters · Keep it friendly.

02
Who's coming over?

Choose how players and spectators find your table.

Public
Open to everyone.

Private
Password-protected.

Lobby password

Required for both players and spectators. Share it with your invited friends.

Maximum players

2
3
4
5
6

Fill empty seats with bots

Dealer bot is always at the table.

Spectators are unlimited.

03
Set a comfortable pace

All values are fictional points, never real money.

Minimum bet (pts)

10

Maximum bet (pts)

250

Minimum balance (pts)

200

Automatically increase the minimum bet

Optional

Increase by (pts)

10

Every (minutes)

15

The first increase happens 15 minutes after creation; increases stop at the maximum bet.

Private

Maya's Island Hangout

MR
Hosted by MayaReef

Created when you open the lobby

Player seats

Up to 6 · Bots on

Bet range

10-250 pts

Entry balance

200 pts minimum

Min. bet increase

+10 pts / 15 min

Spectators

Unlimited · Password required

Create lobby & take a seat

You can invite friends once your table is open.

A good host keeps it easy.

Choose a low minimum for new players. Keep your password handy for private invitations. Empty lobbies become inactive after a few minutes.

Need a hand? Contact admin

Cancel and go back

18+
Free play only. All points are fictional, have no cash value, and cannot be purchased or withdrawn. No cash wagering. Play responsibly, take a break when you need one.

Contact admin · Report issue

Figure 7: Create Lobby: Allows players to create and configure a new game lobby before starting a match.

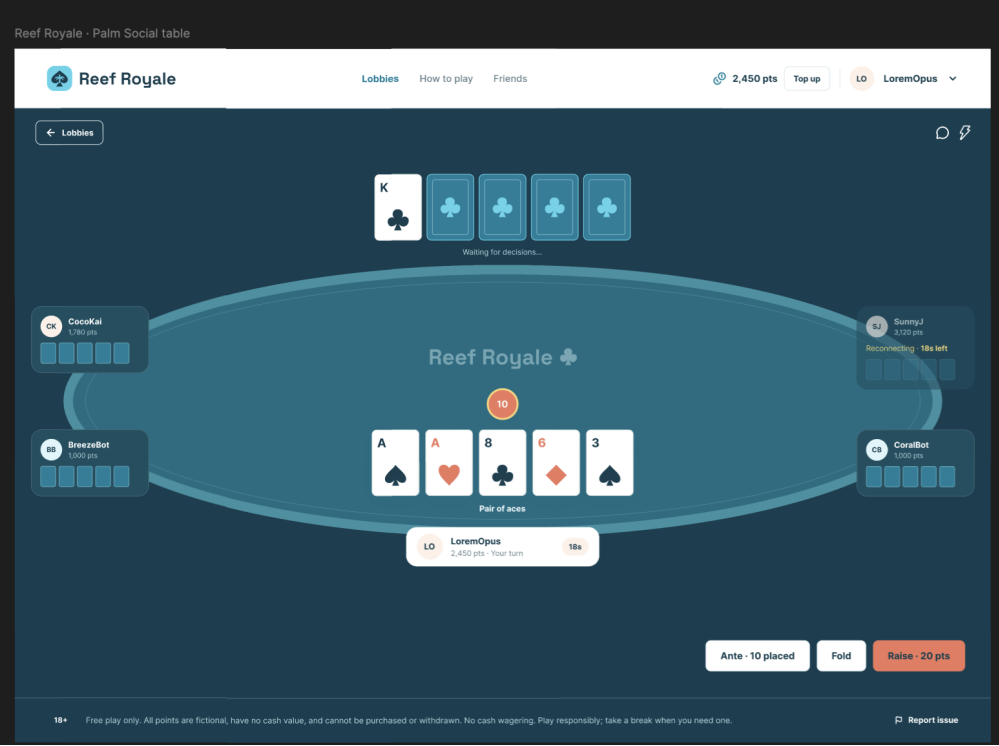


Figure 8: Game Table: The main gameplay interface where players interact during a match.

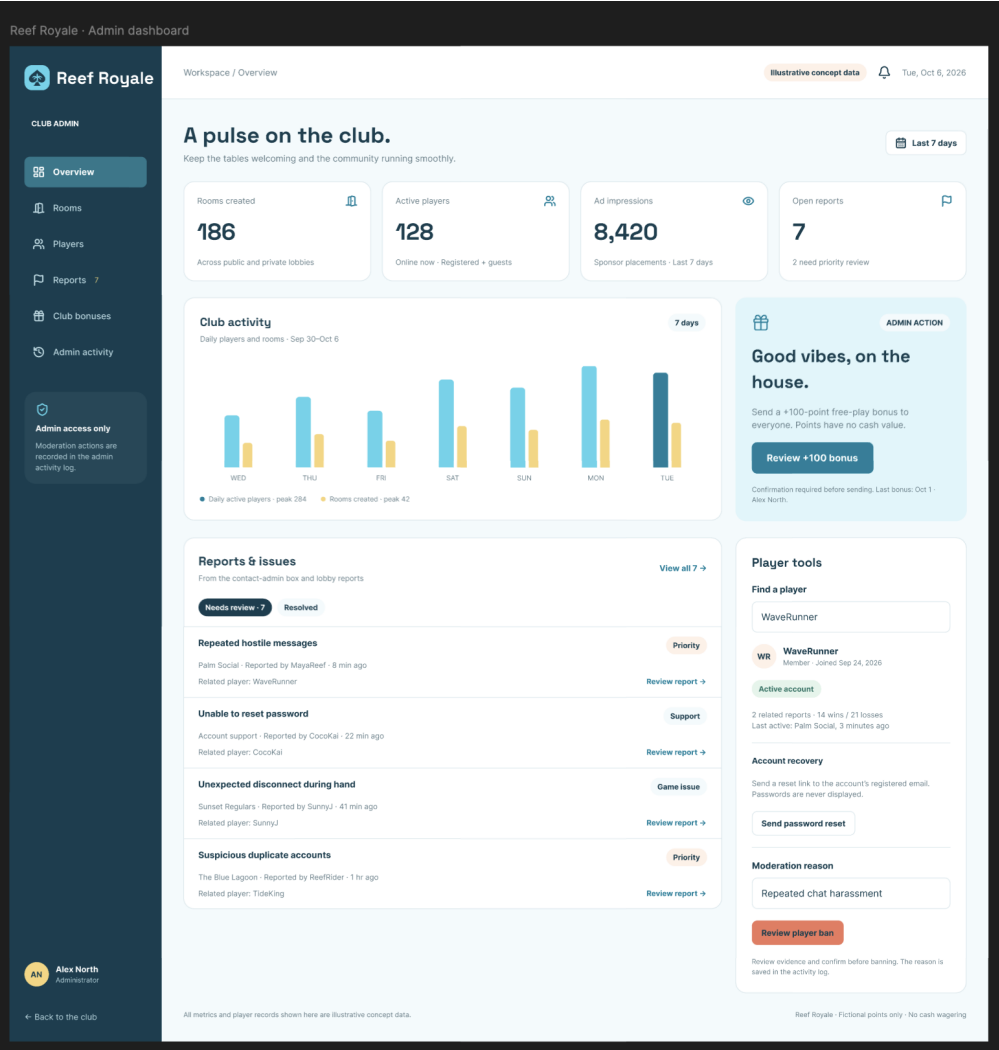


Figure 9: Admin Interface: Administrative controls for managing the application and its features.

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Table 3 shows the iteration planning.

5.2 Iteration/Sprint 1

5.2.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

It.	Dates	Stories	Points	
			Planned	Done
1	01/01 - 02/01	S1 Story Example, S2 Story Example 2	07	05
2	02/01 - 03/01	S3 Story Title, S4 Story Title, S5 Story Title, S6 Story Title	17	15
3	03/01 - 04/01	S7 Story Title, S8 Story Title, S9 Story Title, S10 Story Title, S11 Story Title	21	20
4	04/01 - 05/01	S12 Story Title, S13 Story Title, S14 Story Title, S15 Story Title	19	19
5	05/01 - 06/01	S16 Story Title, S17 Story Title	06	06
Total:			70	65

Table 3: Iteration Planning for Incremental Deliveries

5.2.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.2.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

Figure 10 shows the test coverage

5.2.4 Retroerspective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.3 Iteration/Sprint 2

5.3.1 Planning

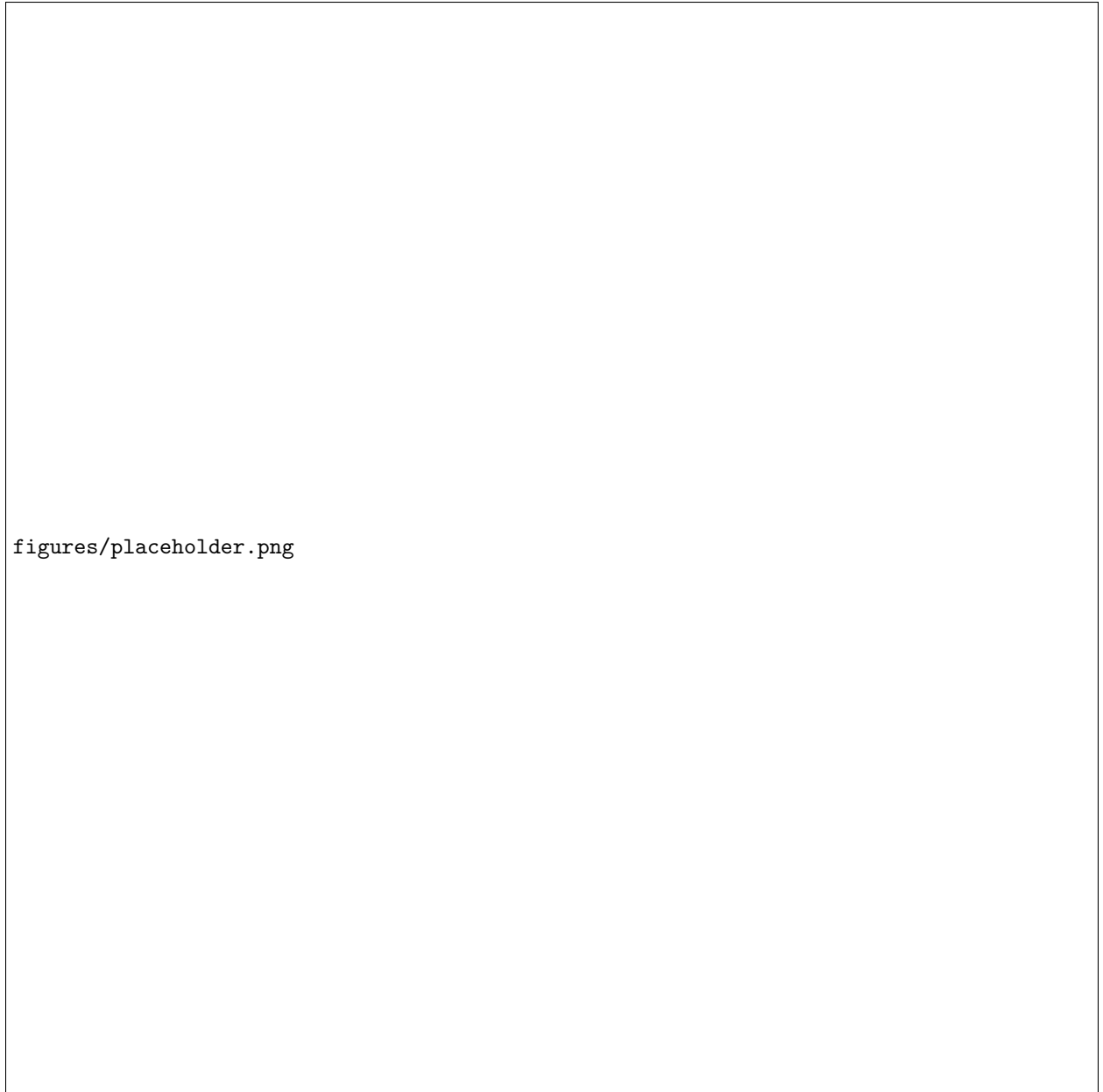
[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]



figures/placeholder.png

Figure 10: Iteration 1 test coverage report

5.3.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482)]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]